Перфоманс без фреймворка: оптимизации, которые работают везде

Илья Салдин | Неконференция | Караганда 2026

Почему перфоманс важен?

-1%

продаж за каждые 100ms задержки (Amazon)

53%

пользователей уходят, если сайт грузится > 3 сек (Google)

SEO

Core Web Vitals влияют на ранжирование с 2021 года

Перфоманс — это не техническая метрика. Это деньги, пользователи и позиции в поиске.

Что оптимизируем?

Vue 3 интернет-магазин электроники

71	11.1s	2.7s	0.52
Lighthouse Score	LCP	FCP	CLS

⚠️ **LCP 11 секунд** — пользователь видит пустой экран дольше, чем готовит кофе

Дорожная карта

01
🔗 Ресурсы
Загрузка ресурсов

02
🖼️ Картинки
Оптимизация изображений

03
🎨 Рендеринг
CSS и рендеринг

04
🚀 Сервер
Серверные оптимизации

Итерация 1: Ресурсы

Проблема: всё блокирует рендеринг

- `<script>` без `defer` блокирует парсинг HTML — браузер останавливает всё, пока скрипт не загрузится и не выполнится
- Шрифты: `font-display: block` — невидимый текст 3 секунды

  (FOIT — Flash of Invisible Text)
- Нет `preload` — браузер сам угадывает приоритеты, и угадывает плохо

Решение: правильный <head>

❌ ДО

```
<link rel="stylesheet"
      href="/assets/main.css">
<script src="/assets/app.js"></script>
<!— Шрифт грузится из CSS —
     браузер узнаёт о нём поздно —>
```

✅ ПОСЛЕ

```
<link rel="preload" href="/hero.jpg"
      as="image">
<link rel="preload"
      href="/fonts/Inter.woff2"
      as="font" crossorigin>
<link rel="stylesheet"
      href="/assets/main.css">
<script defer
        src="/assets/app.js"></script>
<!— + font-display: swap в CSS —>
```

defer — скрипт грузится параллельно, выполняется после парсинга HTML
preload — подсказка браузеру: «это понадобится, начни загрузку сейчас»

Итерация 1: результаты

МЕТРИКА	ДО	ПОСЛЕ	РАЗНИЦА
Score	71	73	+2
FCP	2711 ms	1960 ms	-28%
LCP	11 083 ms	11 335 ms	~0%
Speed Index	2711 ms	1962 ms	-28%

> 💡 FCP улучшился на **28%**, но LCP не изменился — нужно разобраться с картинками!

Итерация 2: Картинки

Проблема: все картинки грузятся сразу

- 20+ картинок начинают грузиться **одновременно** — конкурируют за bandwidth
- Hero-картинка **(LCP element)** конкурирует с остальными — браузер не знает, что она важнее
- `background-image` в CSS — браузер не может оптимизировать приоритет
- Нет `width` / `height` — Layout Shift при загрузке

Решение: приоритеты и lazy loading

```html
<!— LCP-картинка — грузить ПЕРВОЙ —>
<img src="hero.jpg"
     fetchpriority="high"
     width="1200" height="600"
     alt="...">

<!— Остальные — только при скролле —>
<img src="product.jpg"
     loading="lazy"
     width="300" height="300"
     alt="...">
```

fetchpriority="high" — говорит браузеру: «это самая важная картинка, грузи первой»
loading="lazy" — не грузить, пока пользователь не доскроллит
width + height — браузер заранее знает размер, нет layout shift

Итерация 2: результаты

МЕТРИКА	ДО	ПОСЛЕ	РАЗНИЦА
Score	73	80	+7
FCP	1960 ms	1957 ms	~0%
LCP	11 335 ms	4851 ms	**-57%!**
TBT	36 ms	17.5 ms	-51%
CLS	0.52	0	✅

> 🔥 LCP упал на **57%**! `fetchpriority` + `loading="lazy"` — самый большой выигрыш

Итерация 3: Рендеринг

Проблема: браузер рендерит всё подряд

- CSS-файл блокирует первый рендер

  (render-blocking resource — пока CSS не загружен, экран пустой)

- Браузер считает layout для элементов **за пределами экрана** — впустую тратит время
- Изменение одного элемента **пересчитывает layout соседей** — каскадный reflow

Решение: критический CSS и containment

INLINE CRITICAL CSS

```
<style>
  /* Только above-the-fold */
  .hero { ... }
  .header { ... }
</style>
<link rel="stylesheet"
      href="/main.css"
      media="print"
      onload="this.media='all'">
```

CSS CONTAINMENT

```
/* Не рендерить невидимое */
.product-card {
  content-visibility: auto;
  contain-intrinsic-size: 0 300px;
}

/* Изолировать reflow */
.reviews-section {
  contain: content;
}
```

content-visibility: auto — браузер пропускает рендеринг невидимых элементов.
contain: content — reflow внутри блока не затрагивает соседей.

Итерация 3: результаты

МЕТРИКА	ДО	ПОСЛЕ	РАЗНИЦА
Score	80	82	+2
FCP	1957 ms	1955 ms	~0%
LCP	4851 ms	4694 ms	-3%
TBT	17.5 ms	10.5 ms	-40%

> 💡 Маленький выигрыш в LCP, но TBT **-40%**. Рендеринг стал эффективнее — меньше работы у основного потока

Итерация 4: Сервер

Проблема: медленная доставка

- **HTTP/1.1**: head-of-line blocking, максимум 6 параллельных соединений
- **Нет сжатия** — файлы летят в полном размере
- **Нет кеша** — каждый визит как первый
- **Нет Early Hints** — сервер молчит, пока обрабатывает запрос, браузер простаивает

HTTP/2 + Brotli

HTTP/2

- **Мультиплексирование** — все запросы через одно TCP-соединение
- **Бинарное фреймирование** — эффективнее текстового HTTP/1.1
- **Параллельные потоки** — нет head-of-line blocking

Brotli

- На **20–30%** эффективнее gzip
- **Встроенный словарь** для веб-контента (HTML, CSS, JS)
- Quality **4** — баланс скорости сжатия и размера

103 Early Hints + Cache-Control

Обычный HTTP: ⏳ Запрос | Ожидание... | 200 + HTML | 🔌 Загрузка

С Early Hints: 103 💡 | 🔌 Уже грузит! | 200 + HTML | 🔌 Готово

Cache-Control стратегия

ТИП ФАЙЛА	ЗАГОЛОВОК	ЛОГИКА
HTML	`no-cache`	Всегда проверять свежесть
JS/CSS (с хешем)	`immutable, max-age=31536000`	Кешировать навсегда
Картинки	`max-age=86400`	Кешировать на сутки

99

Итерация 4: финальный результат

МЕТРИКА	ДО	ПОСЛЕ	РАЗНИЦА
Score	82	99 🎉	+17
FCP	1955 ms	1206 ms	-38%
LCP	4694 ms	2066 ms	-56%
Speed Index	2491 ms	1206 ms	-52%

🎉 Серверные оптимизации дали **самый большой финальный буст**. HTTP/2 + Brotli + Early Hints + Cache = Score 99

Результат: 71 → 99

МЕТРИКА	BASELINE	ITER 1 РЕСУРСЫ	ITER 2 КАРТИНКИ	ITER 3 РЕНДЕРИНГ	ITER 4 СЕРВЕР
Score	71	73	80	82	99
FCP	2711	1960	1957	1955	1206
LCP	11 083	11 335	4851	4694	2066
SI	2711	1962	1971	2491	1206
TBT	36	36.5	17.5	10.5	10
CLS	0.52	0.521	0	0	0

Чеклист: применить в понедельник

- ✅ Добавьте `defer` к скриптам и `preload` для критических ресурсов
- ✅ `fetchpriority="high"` для LCP-картинки, `loading="lazy"` для остальных
- ✅ Inline critical CSS, `content-visibility: auto` для длинных списков
- ✅ Включите **HTTP/2** и **Brotli** на сервере
- ✅ Настройте `Cache-Control` для разных типов файлов

Всё это работает в **любом фреймворке**: React, Vue, Angular, Svelte, и даже без фреймворка.

Спасибо!

github.com/isaldin/terricon_valley_demo

Telegram: @sald_in • https://sald.in